

INSTRUCCIONES_FIREBASE

Objetivo

Ajustar el flujo de onboarding para que la **fuentes de verdad sea Firestore** y usar `localStorage` solo como fallback temporal cuando Firestore falle.

Cambios implementados

1) `src/services/userService.ts` (nuevo)

Se creó un servicio dedicado de usuario para centralizar lectura/escritura:

- `getUser(userId)`
- Intenta leer primero desde servidor con `getDocFromServer`.
- Si hay falla transitoria/offline, intenta caché con `getDoc`.
- Devuelve `null` solo cuando el documento no existe.
- `updateUser(userId, updates)`
- Usa `setDoc(..., { merge: true })` para persistencia robusta.
- Normaliza payload y guarda compatibilidad de `birthDate / birthdate`.
- Si llega `current_rank`, fuerza consistencia con `onboardingCompleted: true`.
- Devuelve `boolean` de éxito.
- `createUser(user)`
- Crea/mergea el documento de usuario, normaliza email e id.
- Devuelve `boolean` de éxito.

2) `src/services/dbService.ts`

- Se reemplazó la implementación inline de `userService` por re-export:
- `export { userService } from './userService';`
- Así todo el proyecto sigue usando el mismo `userService` centralizado.

3) `src/App.tsx`

Auth (`onAuthStateChanged`)

- Ahora **Firestore es la fuente principal**:
- Si hay documento en Firestore: usa esos datos y decide onboarding según `onboardingCompleted`.
- Si no hay documento: crea usuario base en Firestore (primera vez).
- Solo si Firestore falla (timeout/error): usa fallback local (`localStorage`).
- Regla de onboarding:
- `onboardingCompleted: true` (o Admin) => NO mostrar formulario.
- `onboardingCompleted: false` o usuario nuevo => mostrar formulario.

Al finalizar onboarding (`handleOnboardingComplete`)

- Se arma payload completo y explícito:
- `gender`
- `current_rank`

- `country`
- `state`
- `city`
- `birthDate` + `birthdate` (compat)
- `phone`
- `customAddress`
- `fcmToken`
- `notificationStatus`
- `lastNotificationPromptDate`
- `onboardingCompleted: true`
- Se intenta persistir en Firestore (con timeout).
- Si Firestore falla, se guarda fallback local temporal y se informa por error no bloqueante.

4) `src/components/OnboardingForm.tsx`

- Se agregó envío explícito de `birthdate` además de `birthDate` para compatibilidad.
- El formulario ya enviaba correctamente el resto de campos del perfil.

5) `src/types.ts`

- Se agregó `birthdate?: string` como alias de compatibilidad.

Sobre localStorage (dependencias críticas)

- Los datos críticos del perfil **se leen/escriben en Firestore**.
- `localStorage` queda como **fallback temporal** ante falla de Firestore (red/API).
- El flujo normal, con Firestore habilitado, no depende de localStorage para decidir onboarding.

Validación técnica ejecutada

- `npm run lint` 
- `npm run build` 

Prueba funcional recomendada (cuando Firestore ya esté habilitado)

1. Login con usuario de prueba.
2. Completar onboarding y enviar.
3. Verificar en Firestore (`users/<uid>`) que existan:
 - `onboardingCompleted: true`
 - `gender` , `current_rank` , `country` , `state` , `city` , `birthDate/birthdate` , `phone` .
4. Logout.
5. Login nuevamente.
6. Confirmar que NO aparece onboarding y entra directo al home.